



SOFTWARE ENGINEERING

Ms. Sowmya BP, Archana A & Akshatha PR

Department of Computer Applications

- The broad spectrum of tasks and techniques that lead to an **understanding of requirements** is called **requirements engineering**.
- Requirements engineering builds a **bridge to design and construction**.
- The most **difficult tasks** that a software engineer used to face is **understanding the requirements of a problem**.

- Requirements engineering tasks lead to an understanding of :
 - What the **business impact of software** will be
 - What the **customer wants** and
 - How **end users will interact** with the software
- Software engineers or “**analysts**” and other **project stakeholders** (like managers, customers, end users) **all participate** in requirements engineering

There are **Seven** distinct task in Requirements Engineering.

They are:

1. Inception,
2. Elicitation,
3. Elaboration,
4. Negotiation,
5. Specification,
6. Validation, and
7. Management.

1. Inception—It is a task that defines the scope & nature of the problem to be solved.

- Inception is a task where the requirements engineering **asks a set of questions** to establish a software process.
- In this task, it understands the problem and evaluates with the proper solution.
- It collaborates with the relationship between the customer and the developer.
- The developer and customer decide the overall scope and the nature of the question

SOFTWARE ENGINEERING

Requirements Engineering...



For example:

- ✓ ask a set of questions that establish ...
 - Basic understanding of the problem.
 - The people who want a solution.
 - The nature of the solution that is desired, and
 - The effectiveness of preliminary communication and collaboration between the customer and the developer.

2. Elicitation—

A task that helps stakeholders **define what is required**.

- Elicitation means to find the requirements from anybody.
- The requirements are difficult because the **following problems occur in elicitation**.

(i). Problem of scope: The customer give the unnecessary technical detail rather than clarity of the overall system objective.

(ii) Problem of understanding:

Poor understanding between the customer and the developer regarding various aspect of the project like **capability**, **limitation** of the computing environment.

(iii). Problem of volatility:

In this problem, the requirements change from time to time and it is difficult while developing the project.

3. Elaboration—create an analysis model that identifies data, function and behavioral requirements.

- In this task, the information taken from user during inception and elicitation and are expanded and refined in elaboration.
- Its main task is **developing pure model of software** using functions, feature and constraints of a software.

4. Negotiation—agree on a deliverable system that is realistic for developers and customers

- In negotiation task, a software engineer decides that how will the project be achieved with limited business resources.
- To create rough guesses of development and access the impact of the requirement on the project cost and delivery time.

Eg:

- what are the **priorities** ?
- what is essential ?
- when is it required?

5. Specification

In this task, the requirements engineer constructs a final work product.

- The work product is in the form of software requirements specification.
- In this task, formalize the requirements of the proposed software such as informative, functional and behavioral.
- The requirements are formalize in both graphical and textual formats.

- **In specification**, finally, the problem is specified in some manner and then **reviewed** , can be any one (or more) of the following:
 - A written document
 - A set of models
 - A formal mathematical notations
 - A collection of user scenarios (use-cases)
 - A prototype

6. Validation -

- A Validation is a review mechanism that looks for
 - errors in content or interpretation
 - areas where clarification may be required
 - missing information
 - inconsistencies (a major problem when large products or systems are engineered)
 - conflicting or unrealistic (unachievable) requirements.

6. Validation...

- The work product is built as an output of the requirements engineering and that is **accessed for the quality** through a validation step.
- The formal technical reviews from the software engineer, customer and other stakeholders helps for the primary requirements validation mechanism

7. Requirements management

- It is a set of activities that help the project team to **identify, control and track** the requirements and changes can be made to the requirements at any time of the ongoing project.
- These tasks start with the identification and assign a unique identifier to each of the requirement.
- After finalizing the requirement traceability table is developed.
- The examples of traceability table are the **features, sources, dependencies, subsystems and interface** of the requirement

What is SRS?

- A software requirements specification (SRS) is a **document** that describes **what the software will do** and **how it will be expected to perform**. It also describes the functionality the product needs to fulfill all stakeholders (business, users) needs.
- SRS document describes the **intended purpose, requirements, and nature of software/application/project** to be developed.
- To prepare an SRS document, one should need to have a **functional knowledge** about the project or application, **knowledge of Software/hardware/technology** to be used.

SOFTWARE ENGINEERING

SRS...



- In order to form a [good SRS](#), consider the following points:

1. Introduction

- (i) Purpose
- (ii) Scope
- (iii) Overview

2. General description

3. Functional Requirements

4. Non-Functional Attributes

5. Design Constraints

6. Appendices

Depending upon information gathered after interaction, SRS is developed

1. Introduction:

- **Purpose:** Briefly describe the project and its objectives.
- **Scope:** Define the boundaries of the project. What's included and excluded?
- **Overview** – In this, description of product is explained. It's simply summary or overall review of product

2. General description :

In this, general functions of product which includes **objective of user, a user characteristic, features, benefits, about why its importance** is mentioned. It also describes features of user community

3. Functional Requirements :

- Functional requirements are those which are related to the **technical functionality of the system.**
- Functional requirements are the **product features** or **its functions** that must be designed directly for the users and their convenience.
- They define the functionality of the software, which the software engineers have to develop so that the users could easily perform their tasks up to the business **requirements**

4. Non-functional requirement:

This is a **requirement** that specifies criteria that can be used to judge the operation of a system in particular conditions, rather than specific behaviors.

Nonfunctional Requirements (NFRs) define system attributes such as **security, reliability, performance, maintainability, scalability, and usability**

5. Design Constraints :

In this, constraints which simply means limitation or restriction are specified and explained for design team. Examples may include use of a particular algorithm, hardware and software limitations, etc.

6. Appendices :

In this, additional information like **references** from where information is gathered, definitions of some **specific terms, acronyms, abbreviations**, etc. are given and explained.

Functional and Non-functional Requirements

- **Functional requirements** are those which are related to the **technical functionality** of the system.
- **Non-functional requirement** is a **requirement** that specifies **criteria** that can be used to judge the operation of a system in particular conditions, rather than specific behaviors.

Functional and Non Functional Requirements

- **Functional requirements** are the **product features** or **its functions** that must be designed directly for the users and their convenience. They define the functionality of the software, which the software engineers have to develop so that the users could easily perform their tasks up to the business **requirements**
- **Nonfunctional Requirements (NFRs)** define system attributes such as **security, reliability, performance, maintainability, scalability, and usability**.

EXAMPLES OF

FUNCTIONAL REQUIREMENTS

VS

NON-FUNCTIONAL REQUIREMENTS

The vehicle must be fast



The engine in the vehicle should be able to reach at least 150 MPH

The vehicle must be red



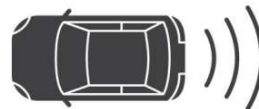
The vehicle should be painted using House of Kolor - KBC11 - Apple Red

The vehicle must be safe for drivers



The vehicle and all its parts should comply with all Federal Motor Vehicle Safety Standards (FMVSS)

Rear camera in the automobile should detect a threat or object



Rear camera must notify the driver of a threat or object within .2 seconds of the detection

Case Study 1: Online Shopping System (E-Commerce Website)

An organization wants to develop an online shopping platform similar to Amazon, where customers can browse products, place orders, and track deliveries.

Functional Requirements (What the system must do)

1. The system shall allow users to register and log in.
2. The system shall allow users to search products by name, category, and price.
3. The system shall allow users to add products to a shopping cart.
4. The system shall allow users to place an order and make online payments.
5. The system shall allow users to track order status.
6. The system shall allow administrators to add, update, or remove products.

Non-Functional Requirements (How the system performs)

1. **Performance:** Product search results must be displayed within 2 seconds.
2. **Security:** User passwords must be **encrypted**.
3. **Availability:** The system must be available **99.9% of the time**.
4. **Scalability:** The system must support **at least 10,000 concurrent users**.
5. **Usability:** Users should be able to place an order in **no more than 5 steps**.

Case Study 2: Online Banking System

A bank wants to provide customers with an online banking application for account management and transactions.

Case Study 3: University Course Registration System

A university wants an online system for students to register for courses each semester.

Establishing the Ground Work

The steps required to establish the groundwork for an **understanding of software requirements** are:

1. Identifying Stakeholders--Stakeholder can be “**anyone who benefits** in a direct way or indirect way from the system which is being developed”

- Ex. Business manager, project manager, marketing people, software engineer, support engineer, end-users, internal-external customers, consultants, maintenance engineer.
- Each one of them has **different view** of the system

2. Recognizing Multiple Viewpoints

- Many different stakeholders exist, the requirements of the system will be explored from **many different people's point of view**.
 1. **Marketing group** concern about feature and function to excite potential market. To **sell easily** in the market.
 2. **Business manager** concern about feature built **within budget** and will be ready to meet market.
 3. **End user** – Easy to **learn and use**.
 4. **Software Engineer** – **product functioning** at various infrastructure support.
 5. **Support engineer** – **Maintainability of software**.
- Role of Requirement Engg. is to **categorize** all stakeholder information in a way that there could be no inconsistent or conflict requirement with one another.

3. Working towards Collaboration

- Requirements Engg. identify **areas of commonality** (i.e. Agreed requirement) and **areas of conflict** or **inconsistency**.
- **Business manager** or **senior technologist** may make final decision.

4. Asking the Questions

- The first set of context-free questions focuses on the customer and other stakeholders, the overall **project goals** and **benefits**.
 - Who is behind the request for this work?
 - Who will use the solution?
 - What will be the economic benefit of a successful solution?
 - Is there another source for the solution that you need?
- These questions will help to know— stakeholder's interest in the software & measurable benefit of successful implementation.

Asking the Questions...

- **Next set of questions** – **better understanding of the problem.**
 - What business problem(s) will this solution address?
 - Describe business environment in which the solution will be used?
 - Will performance or productivity issues affect the solution?

Final set of questions – **Effectiveness of communication**

- Are my questions relevant to the problem?
- Am I asking too many questions?
- Can anyone else provide additional information?
- should I be asking you anything else?

- Requirements elicitation (also called **requirements gathering**) combines **elements of problem solving, elaboration, negotiation, and specification.**

Approach for eliciting requirement:

- 1. Collaborative Requirement Gathering**
- 2. Quality Function Deployment**
- 3. User (Usage)Scenarios**
- 4. Elicitation Work Products**

- Collaborative Requirement gathering have some [basic guidelines](#):
 - **Meetings** are conducted & attended by both software engineer & interested stakeholders.
 - **Rules** for preparation and participation are **established**.
 - **Agenda** should be **formal** enough to cover all important points, but informal enough to encourage the free flow of ideas.
 - A **facilitator** (customer / developer / outsider) **controls** the meeting.
 - A “**definition mechanism**” (work sheets, flip charts, wall stickers, chat room , etc.) is used.

- **During the meeting:**
 - The problem is identified.
 - Elements of the solution are proposed.
 - Different approaches are negotiated.
 - A preliminary set of solution requirements are obtained.
 - The atmosphere is collaborative and non-threatening.

- Flow of event – Outline the **sequence of events** occurs
 - Requirement gathering meeting (initial meeting)
 - During meeting
 - Follow the meeting
- In initial meeting, distribute “**Product request**” (defined by stakeholder) to all attendee.

- Based on product request, each attendee is asked to make
 - **List of objects** (Internal or external system objects)
 - **List of services**(Processes or functions)
 - **List of constraints** (cost, size, business rules) and **performance criteria** (speed, accuracy) are developed.

SOFTWARE ENGINEERING

1. Collaborative Requirements Gathering...



- **Collect** lists from everyone and **combine**.
- Combined list **eliminates redundant entries**, **add new ideas**, but does not delete anything.
- Objective is to develop a consensus (process of decision making) list in each topic area (objects, services, constraints and performance).
- **Based on lists, team is divided into smaller sub-teams** : each works to develop mini-specification for one or more entries on each of the lists.

SOFTWARE ENGINEERING

1. Collaborative Requirements Gathering...



- Each sub-team presents its mini-specification to all attendees for discussion.
 - **Addition**, **deletion** and further **elaboration** are made.
- Now each team makes a **list of validation criteria** for the product and present to team.
- Finally, one or more participants is assigned the task of writing a **complete draft specification**.

- It is a technique that **translate the needs of the customer** into **technical requirement** for software.
- Concentrates on **maximizing customer satisfaction**.
- QFD emphasizes – what is valuable to the customer and then deploys these values throughout the engineering process.

QFD identifies **Three** types of requirements:

1. Normal Requirements –

- The **objectives and goals** that are stated for a product or system during meetings with the customer. If these requirements are present, the customer is satisfied.
- Examples of normal requirements might be requested types of **graphical displays, specific system functions, and defined levels of performance.**

2. Expected Requirements –

Customer **does not explicitly** state them. Customer assumes it is implicitly available with the system.

- These requirements are implicit to the product or system and may be so fundamental that the customer does not explicitly state them.
- Their absence will be a cause for **significant dissatisfaction**.
- Examples of expected requirements are:
 - Ease of human/machine interaction,
 - overall operational correctness and reliability
 - ease of software installation , etc..

3. Exciting Requirements-

Features that go **beyond the customer's expectation.**

- Ex--software for a new mobile phone comes with standard features, but is coupled with a set of unexpected capabilities (e.g., multitouch screen, visual voice mail, screen recording etc.) **that delight** every user of the product.

-
- Till the software team does not understand how the features and function are used by the end users it is difficult to move technical activities.
 - To achieve above problem the software team produces a set of structure that identify the usage for the software.
 - This structure is called as '**Use Cases**'

- As requirements are gathered as part of informal meetings (like QFD), the software engineer (analyst) can **create a set of scenarios** that identify a thread of **usage for the system** to be constructed.
 - The scenarios, often called **use-cases**, provide a description of how the system will be used.
- A **scenario** is a **scene** that illustrates **some interaction** with a proposed system.
- Developers and users **create a set of usage threads** for the system to be constructed.

-
- A use case scenario is a story about how someone or something **external** to the software (known as an **actor**) **interacts with the system**.
 - i.e., it describes **how the system will be used**.
 - Each scenario is described from the point-of-view of an “actor”—a person or device that interacts with the software in some way.

- Elicitation **work product** will vary **depending upon the size** of the system or product to be built.
- For most systems, the work products include:
 - Statement of **need** and **feasibility**.
 - Statement of **scope**.
 - List of **participants** in requirements elicitation.
 - Description of the system's technical **environment**.
 - List of requirements and associated **domain constraints**.
 - List of usage **scenarios**.
 - Any **prototypes** developed to refine requirements.

- A use case is a **methodology** used in system **analysis** to **identify**, **clarify**, and **organize** system requirements.
- The use case is made up of a **set of possible *sequences of interactions between systems and users*** in a particular environment and related to a particular goal

- Use case tells a **stylized story** about **how an end user interacts with the system** under a specific set of circumstances.
- Regardless of its form, a use case **depicts the software** or system from the **end user's point of view**.

-
- Use case tells a **stylized story** about **how an end user interacts with the system** under a **specific set of circumstances**.
 - Regardless of its form, a use case **depicts the software** or system from the **end user's point of view**.

- **Step One** – Define the set of actors that will be involved in the story
 - **Actors** are **people, devices, or other systems** that use the system or product **within the context of the function** and behavior that is to be described
 - Actors are those that **communicate with the system** or product and that are external to the system itself
- **Step Two** – Develop use cases, where each one answers a set of questions

SOFTWARE ENGINEERING

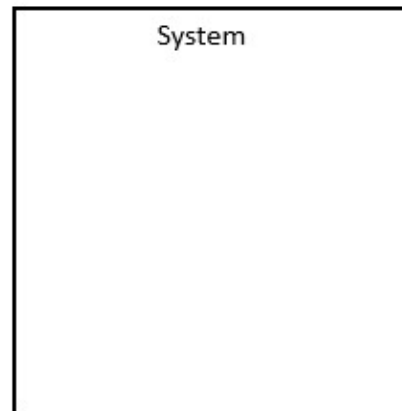
Developing Use Cases...

- Components of Use Case diagram

- Use case:



- Subsystem:



- Actor:



- Association:



- An use case involves **a sequence of messages** among the system and its actors. This can be repeated several times.

Consider for example **Hotel room booking system**

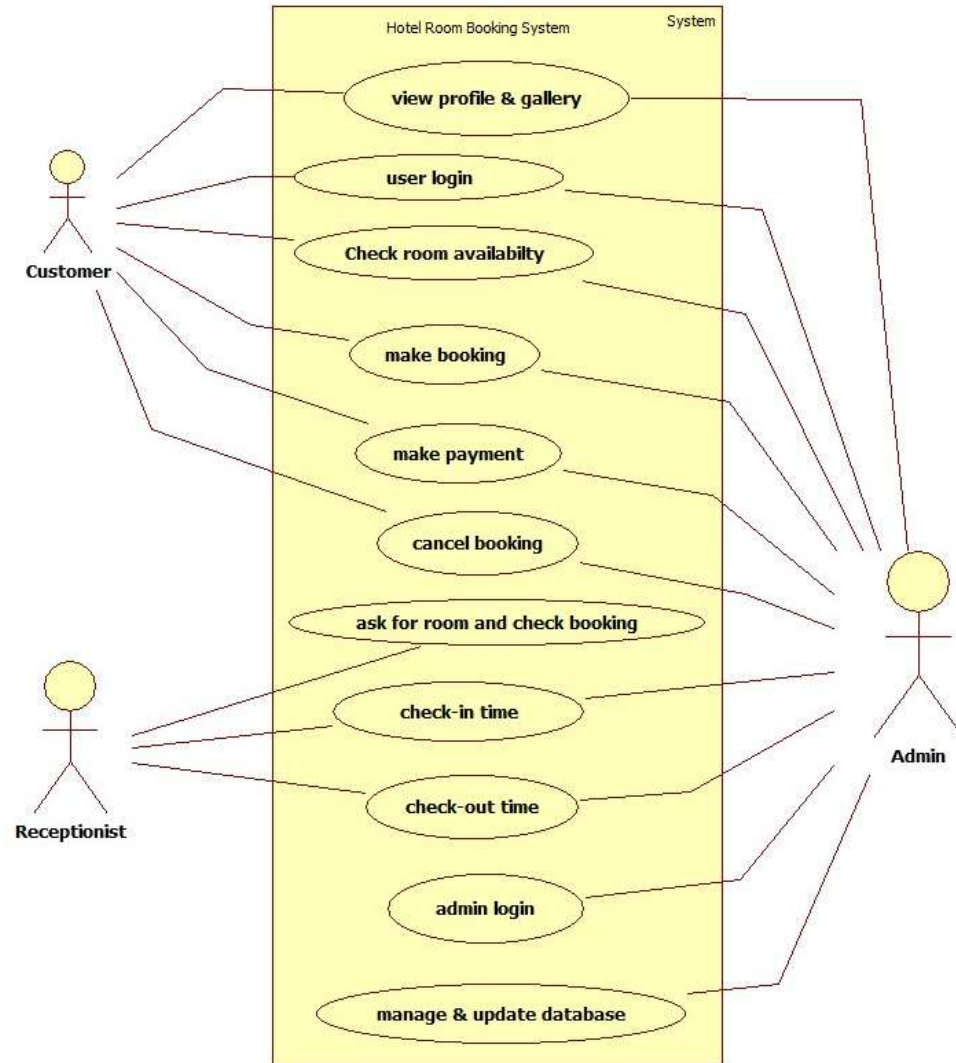
Actors: Customer, Receptionist, Administrator

Use Cases:

- **Customer's** functionalities are → User login, check room availability, book room, cancel booking, make payment.
- **Receptionist** → ask for room booking, check booking, check-in time, check-out time
- **Admin** → Manage users, login, manage database, admin manages all the functionalities of the application

SOFTWARE ENGINEERING

Developing Use cases...



Experiential Learning



Draw an Use case diagram for the following applications

1. Online shopping app (eg: amazon)
2. Food ordering app (eg: swiggy)
3. Travel booking application (eg: makemytrip)
4. Pay your electricity bill in bescom app (eg: e-billing)
5. Platform ticket generation system

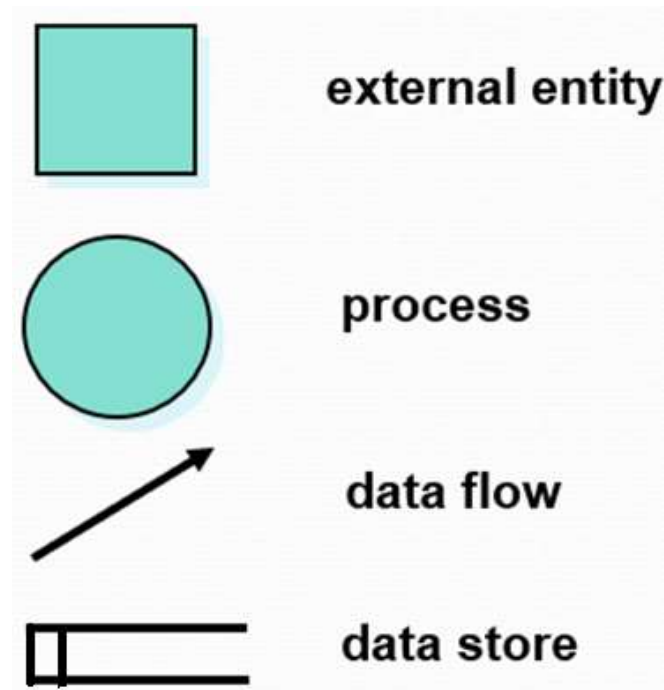
- Data flow modeling is a core modeling activity in **structured analysis**.
- Flow oriented modeling represents **how data objects are transformed** when they move through the system.
- **Data flow diagram (DFD)** is the diagrammatic form that is used to represent the **data flow**.
- The purpose of DFD is to **provide a semantic bridge between user and system's developers**

Data Flow Diagram:

- Usually **begin** with drawing a **context diagram**, a simple representation of the whole system.
- To **elaborate further** from that, drill down to a **level 1 diagram** with additional information about the **major functions** of the system.
- This could continue to evolve to become a **level 2** diagram when further analysis is required.
- Progression to level 3, 4 and so on is possible

Creating a Data Flow Model

- Notations used in flow oriented model are as follows:



SOFTWARE ENGINEERING

Flow Oriented Modeling – DFD...



External Entity:



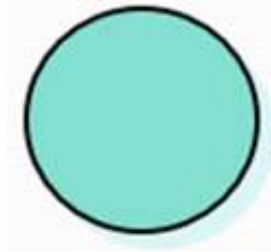
- Producer or consumer of data
- Data must always originate somewhere and must always be sent to something
- For Example: Computer based system

SOFTWARE ENGINEERING

Flow Oriented Modeling – DFD...



Process:



- **A data transformer**
(changes input to output)
- **For Eg:** compute taxes, determine area, format report, display graph
- Data must always be processed in some way to achieve system function.

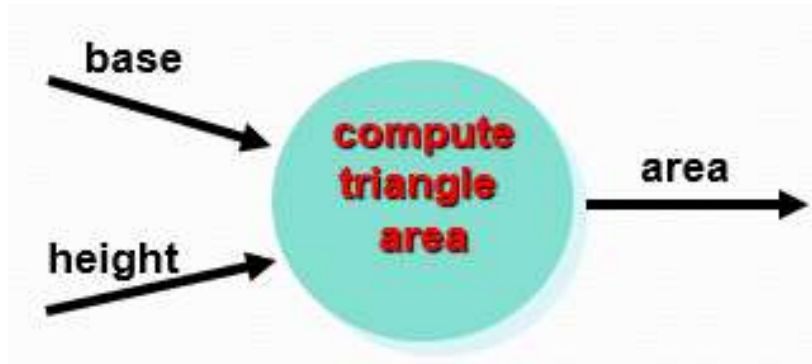
SOFTWARE ENGINEERING

Flow Oriented Modeling – DFD...

Data Flow:



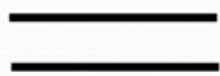
- Data flows through a system, beginning as input and transformed into output.



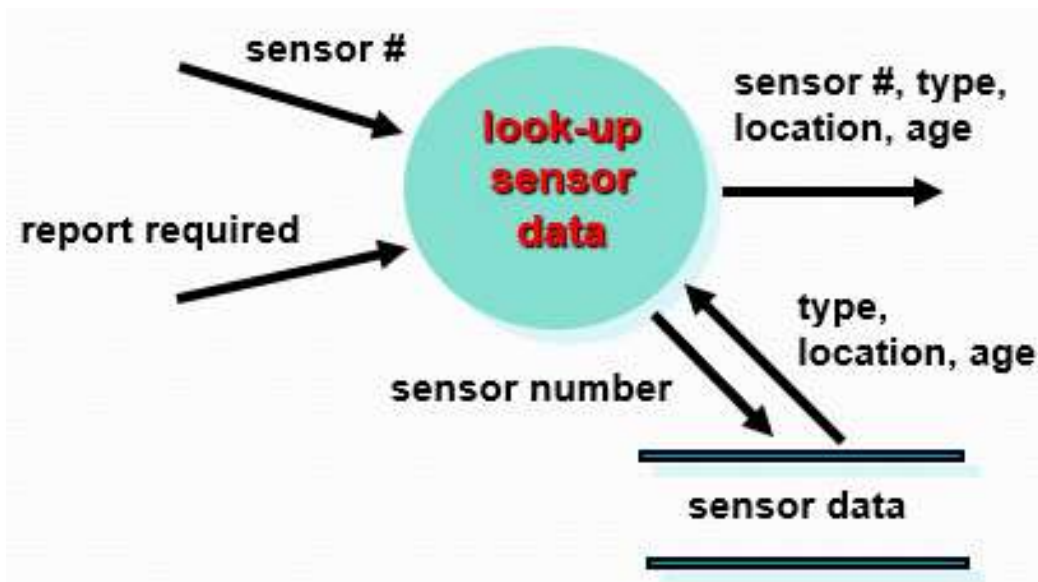
SOFTWARE ENGINEERING

Flow Oriented Modeling – DFD...

Data Stores:



- Data is often stored for later use.



DFD:

Guidelines:

1. The level-0 DFD should depict the software / system as a **single bubble**
2. Input and output should be clearly noted
3. Refinement should begin by isolating **candidate processes, data objects, and data stores** to be represented at the **next level**
4. All arrows and bubbles (processes) **must be labeled** with meaningful names
5. Information flow continuity must be maintained from level to level

DFD rules and tips

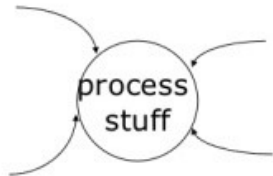
- Each process should have at least one input and an output.
- Each data store should have at least one **data flow** in and one **data flow** out.
- Data stored in a system must go through a process.
- All processes in a **DFD** go to another process or a data store.

- When creating data flow diagrams (DFD's), there are certain rules which must be followed. These rules allow for the DFD to make sense and also to be easily understood. The rules are:
 - 1. All data flows must flow to or from a process**
 - All flows of data must be either coming from or going to a process.
 - External entities can not flow directly to each other.
 - A data flow can not link a data store to an external entity. Data can not move between data stores without first being processed.

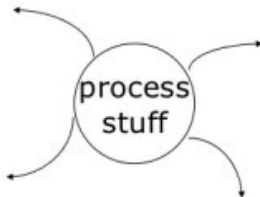
2. A Process must have at least one input flow and one output flow.

- When a process has input flow but no output flow, it is called a “**black hole**”. When a process has output flows but no input flows, it is called a “**miracle**”.

Data Flow Diagram Don'ts



1. 'Black Hole'



2. 'It's a Miracle'

3. The inputs to a process must be sufficient to produce output flows.

- A “**grey hole**” is when the outputs of a process are greater than the sum of its inputs. For example, if a customers name and address is an input, their bank details can not be an output, as the process doesn’t have enough information to produce it.

4. Processes must transform data.

- When naming data flows, adjectives should be used which show how processing has changed the data flow.

5. Data Flows can not cross each other.

- The flows of data can not cross each other. To overcome this problem, data stores and entities can be duplicated.
- However, processes can not be duplicated. Data flows must be unidirectional.

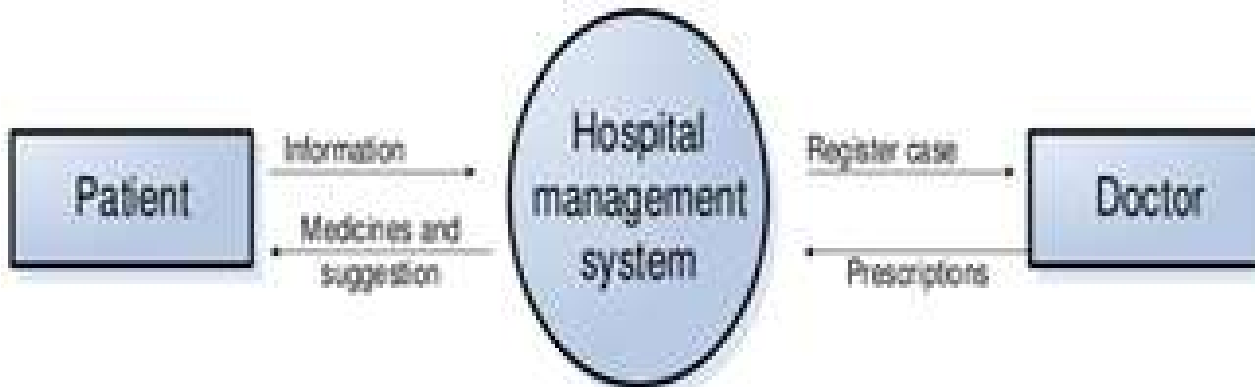
6. Entities must be labelled in lower case.

SOFTWARE ENGINEERING

Creating a Data Flow Model

Level-0 DFD:

Example-1: Hospital Management System

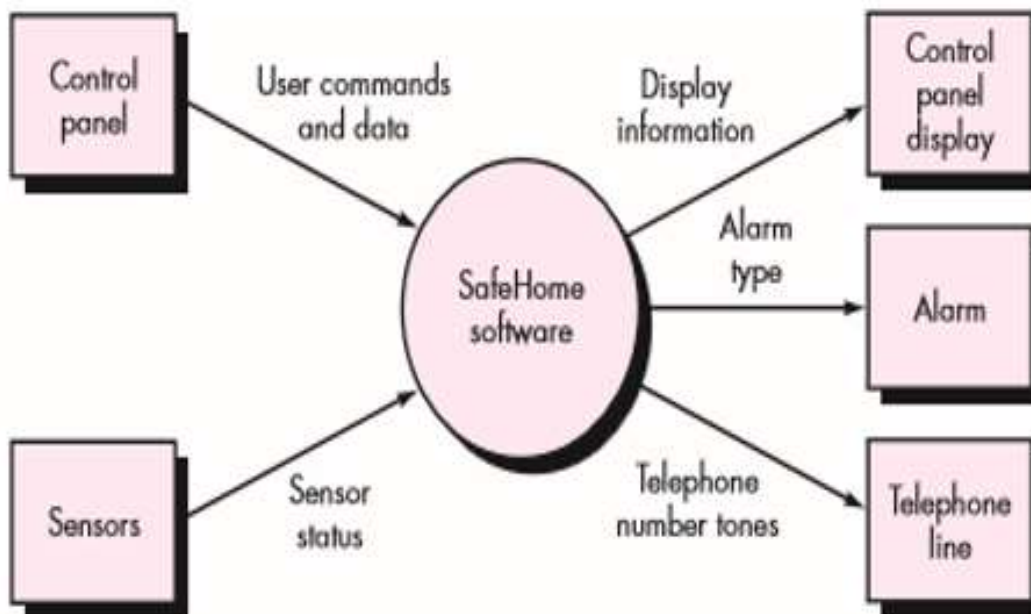


SOFTWARE ENGINEERING

Creating a Data Flow Model

Level-0 DFD:

Example-2: Safety home security system



Constructing a Level-1 DFD

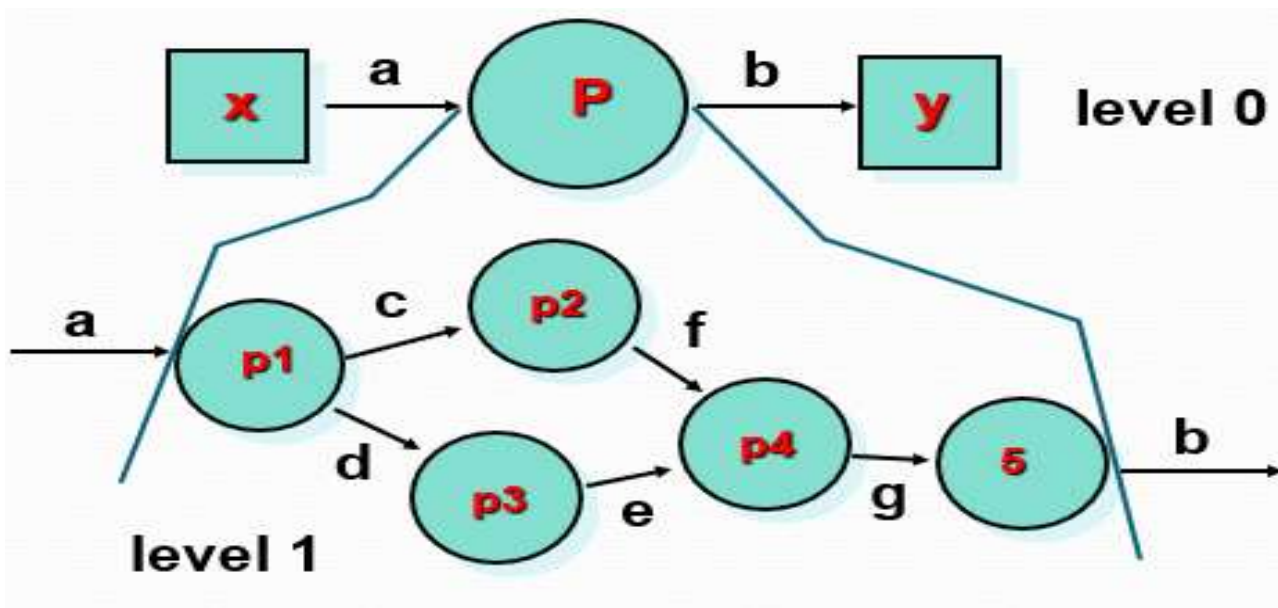
- Write a narrative describing the transform
- Parse to determine next level transforms
- “Balance” the flow to maintain data flow continuity
- Develop a level -1 DFD
- Use a **1:5** (approx.) expansion ratio

SOFTWARE ENGINEERING

Flow Oriented Modeling

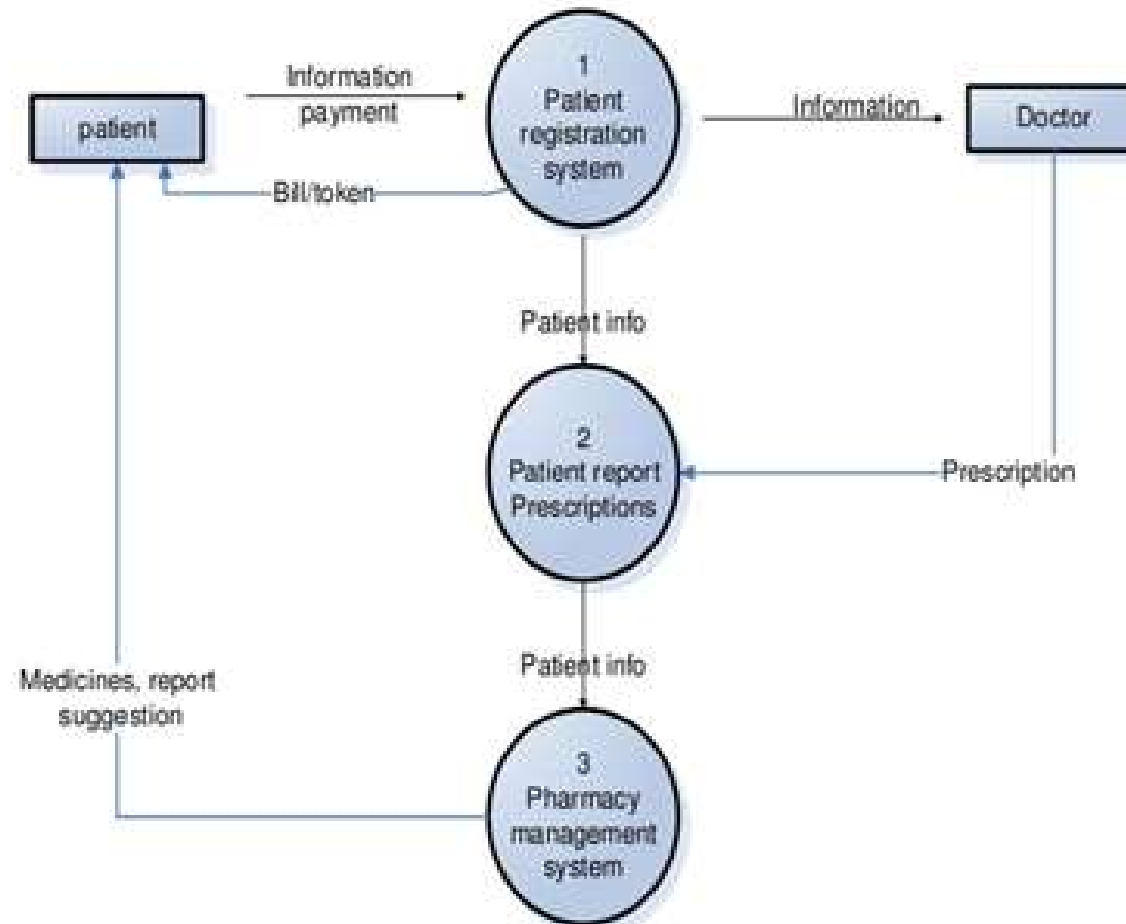
Constructing a Level-1 DFD

- Example: Dataflow hierarchy



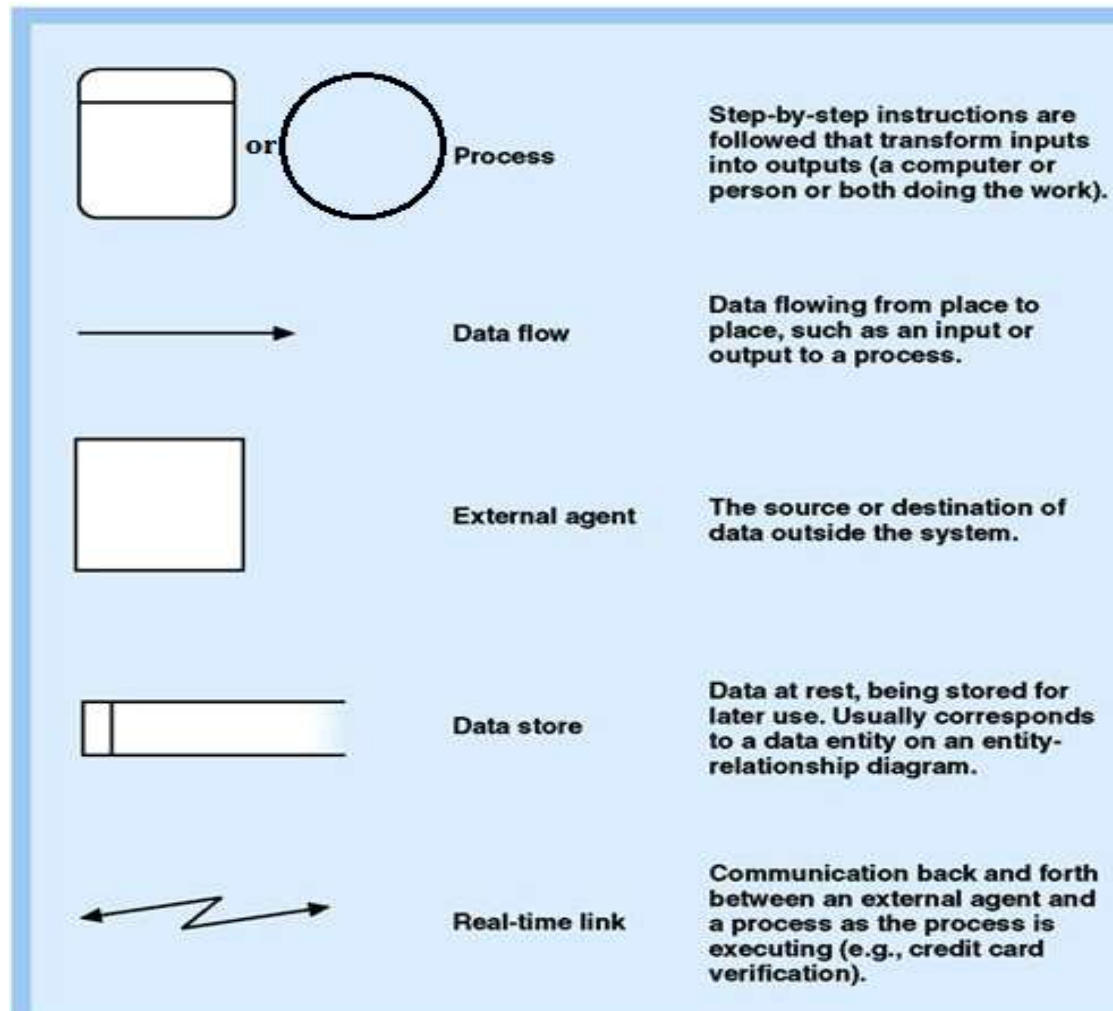
Hospital Management System

- Level-1 DFD



SOFTWARE ENGINEERING

Notations used in DFD diagram - summary

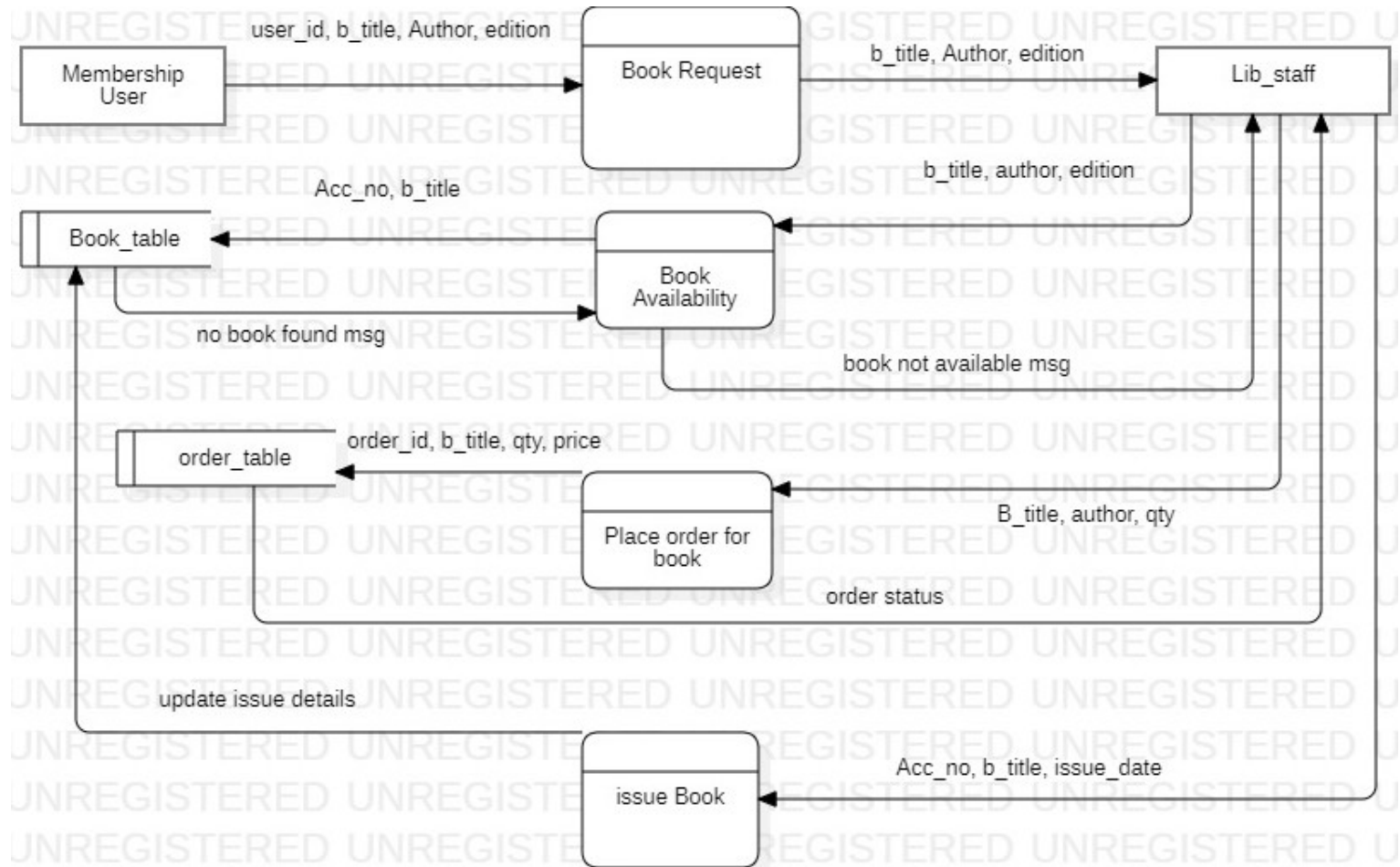


- Each bubble is refined until it does just one thing
- The **expansion ratio decreases as the number of levels increase.**
- Most systems require between 3 and 7 levels for an adequate flow model
- A single data flow item (arrow) may be expanded as levels increase (data dictionary provides information)

-
- Draw Level-0, level-1 and level-2 DFD for Library Management System

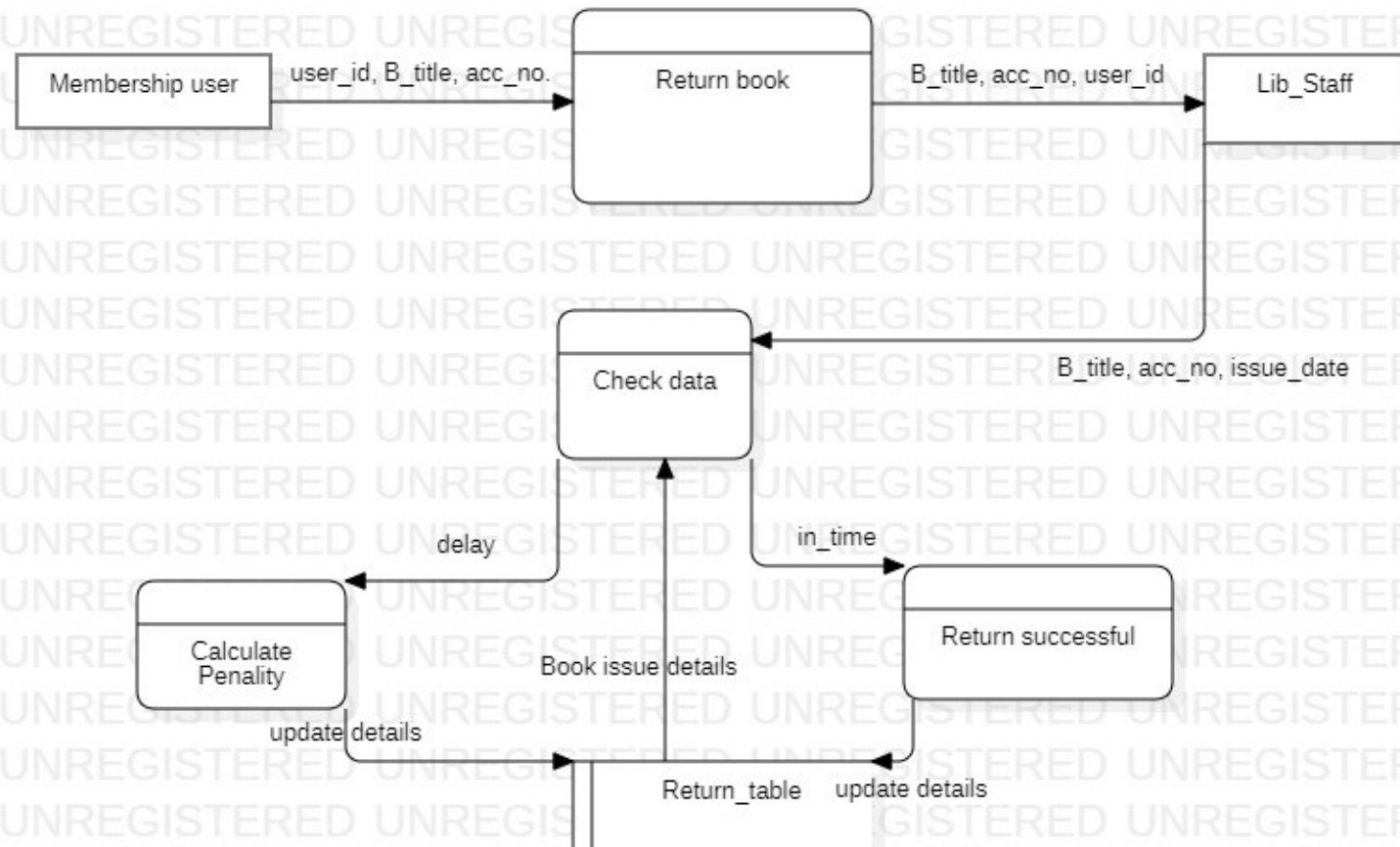
Library Management System

- Level-1 DFD



Library Management System

- Level-2 DFD



Exercise 1: Online Bookstore System

Scenario:

An online bookstore allows customers to browse books, place orders, and make payments. The system also manages inventory and generates reports for the admin.

Task:

1. Draw a **Level 0 DFD** (Context Diagram) for the online bookstore system.
2. Draw a **Level 1 DFD** breaking down the "Process Order" function.

Exercise 2: Hospital Management System

Scenario:

A hospital management system allows patients to book appointments, doctors to update medical records, and admins to generate reports.

Task:

1. Draw a **Level 0 DFD** (Context Diagram) for the hospital management system.
2. Draw a **Level 1 DFD** breaking down the "Book Appointment" function.

.

Exercise 3:. Online Food Delivery System

Scenario:

An online food delivery system allows customers to browse restaurants, place orders, and track deliveries. Restaurants receive orders and update the system with delivery status.

Task:

1. Draw a **Level 0 DFD** (Context Diagram) for the online food delivery system.
2. Draw a **Level 1 DFD** breaking down the "Place Order" function.

Exercise 4:. Student Enrollment System

Scenario:

A student enrollment system allows students to enroll in courses, professors to update course details, and admins to generate enrollment reports.

When a student registers for a course, the system checks if there is room in the course. The system then notifies the student of their registration status. If the student is successfully enrolled, the system sends an enrollment notice to the online learning platform, which updates the course information. Additionally, the faculty member is notified that a student has been added to the course.

Task:

1. Draw a **Level 0 DFD** (Context Diagram) for the student enrollment system.
2. Draw a **Level 1 DFD** breaking down the "Enroll in Course" function.



THANK YOU

Ms. Sowmya BP, Archana A & Akshatha PR

Department of Computer Science

archana@pes.edu

+91 80 6666 3333 Extn 392